

Deep Learning

Loading and Preprocessing Data with the Tensorflow Data API

G. Marques

So far we have used only datasets that fit in memory or have resorted the Keras `ImageDataGenerator` to read images from disk. In Deep Learning systems are often trained on very large datasets that will not fit in RAM memory. Also most datasets will not conform with rigid directory structure needed for the Keras `ImageDataGenerator` - for this generator, the data must be divided in three directories, `train/`, `validation/` and `test/`, and each directory must be divided in sub-directories with the examples of each class.

The `tf.data` API enables us to deal with all types of data, where to get it and how to transform it. Tensorflow takes care of the implementation details such as multithreading, batching, shuffling, prefetching and chaching.

Check out some official resources:

- [TensorFlow data API](#)
- [tf.data.Dataset](#)
- [Better performance](#)
- [Dealing with CSV data](#)

The Data API

The `tf.data` API revolves around the concept of a `tf.data.Dataset` abstraction that represents a sequence of elements, in which each element consists of one or more components. For example, in an image pipeline, an element might be a single training example, with a pair of tensor components representing the image and its label.

Simple Example:

The function `tf.data.Datasets.from_tensor_slices()` takes a tensor and creates a `tf.data.Dataset` with the tensor elements (along the first dimension):

```
X=tf.range(10)
print(X)
DT=tf.data.Dataset.from_tensor_slices(X)
print(DT)
for i in DT:
    print(i)
```

	output
<code>tf.Tensor([0 1 2 3 4 5 6 7 8 9], shape=(10,), dtype=int32)</code>	
<code><TensorSliceDataset shapes: (), types: tf.int32></code>	
<code>tf.Tensor(0, shape=(), dtype=int32)</code>	
<code>tf.Tensor(1, shape=(), dtype=int32)</code>	
<code>tf.Tensor(2, shape=(), dtype=int32)</code>	
<code>:</code>	
<code>tf.Tensor(9, shape=(), dtype=int32)</code>	

The Data API

The `tf.data` API revolves around the concept of a `tf.data.Dataset` abstraction that represents a sequence of elements, in which each element consists of one or more components. For example, in an image pipeline, an element might be a single training example, with a pair of tensor components representing the image and its label.

Simple Example:

The function `tf.data.Datasets.from_tensor_slices()` takes a tensor and creates a `tf.data.Dataset` with the tensor elements (along the first dimension):

```
X=tf.convert_to_tensor(numpy.random.randint(0,10,(3,5),dtype='int32'))
print(X)
DT=tf.data.Dataset.from_tensor_slices(X)
print(DT)
for i in DT:
    print(i)
```

	output
<code>tf.Tensor([[6 5 7 4 7]</code>	
<code> [4 3 6 6 9]</code>	
<code> [9 7 2 6 7]], shape=(3, 5), dtype=int32)</code>	
<code><TensorSliceDataset shapes: (5,), types: tf.int32></code>	
<code>tf.Tensor([6 5 7 4 7], shape=(5,), dtype=int32)</code>	
<code>tf.Tensor([4 3 6 6 9], shape=(5,), dtype=int32)</code>	
<code>tf.Tensor([9 7 2 6 7], shape=(5,), dtype=int32)</code>	

The Data API

The `tf.data` API revolves around the concept of a `tf.data.Dataset` abstraction that represents a sequence of elements, in which each element consists of one or more components. For example, in an image pipeline, an element might be a single training example, with a pair of tensor components representing the image and its label.

Simple Example:

The function `tf.data.Datasets.from_tensor_slices()` takes a tensor and creates a `tf.data.Dataset` with the tensor elements (along the first dimension):

One often wants to look at just a few items in the dataset. For that we can use the `take()` method.

```
X=tf.range(10)
print(X)
DT=tf.data.Dataset.from_tensor_slices(X)
print(DT)
for i in DT.take(2):
    print(i)
```

output

```
tf.Tensor([0 1 2 3 4 5 6 7 8 9], shape=(10,), dtype=int32)
<TensorSliceDataset shapes: (), types: tf.int32>
tf.Tensor(0, shape=(), dtype=int32)
tf.Tensor(1, shape=(), dtype=int32)
```

The Data API

The `tf.data` API revolves around the concept of a `tf.data.Dataset` abstraction that represents a sequence of elements, in which each element consists of one or more components. For example, in an image pipeline, an element might be a single training example, with a pair of tensor components representing the image and its label.

Simple Example:

The function `tf.data.Datasets.from_tensor_slices()` takes a tensor and creates a `tf.data.Dataset` with the tensor elements (along the first dimension):

One can also look at the dataset items by explicitly creating a Python iterator and using the `next()` function.

```
X=tf.range(10)
DT=tf.data.Dataset.from_tensor_slices(X)
it=iter(DT)
print(it)
print(next(it))
print(next(it))
```

output

```
<tensorflow.python.data.ops.iterator_ops.OwnedIterator object at
0x7fef4ddf8898>
tf.Tensor(0, shape=(), dtype=int32)
tf.Tensor(1, shape=(), dtype=int32)
```

Pipelining Transformations

The `tf.data.Dataset` API supports all sorts of transformations, enabling the construction of efficient and descriptive pipelines. Each transformation returns a new dataset. The usage follows this common pattern:

- 1 Create a source dataset from data (stored in memory or in disk).
- 2 Create new datasets by applying data processing transformations.
- 3 Use the last created dataset to iterate and process its elements

Some Functions for Dataset Creation

- `tf.data.Dataset.from_tensor_slices`
Creates a dataset from a tensor or a Python list.
- `tf.data.TextLineDataset(['file1.txt', 'file2.csv'])`
Creates a dataset from text lines in one or more files
- `tf.data.Dataset.list_files('/path/*.txt*')`
Creates a dataset of all files matching a pattern.
- `tf.data.Dataset.from_generator`
Creates a dataset from the elements returned by a generator function.
- `tf.data.TFRecordDataset(['file1.tfrecords', 'file2.tfrecords'])`
A dataset from data written in a TFRecord format.

Pipelining Transformations

The `tf.data.Dataset` API supports all sorts of transformations, enabling the construction of efficient and descriptive pipelines. Each transformation returns a new dataset. The usage follows this common pattern:

- 1 Create a source dataset from data (stored in memory or in disk).
- 2 Create new datasets by applying data processing transformations.
- 3 Use the last created dataset to iterate and process its elements

Some Functions for Pipeline Execution

- **repeat method:**
The `repeat(Ntimes)` method returns a new dataset with the elements of the original one repeated `Ntimes`. If called with no arguments, the source dataset is repeated indefinitely.
- **batch method:**
The `batch(bSize)` method groups the items in the source dataset in batches of `bSize`.
- **shuffle method:**
The `shuffle(buffer_size=Nitems)` method fills a buffer with `Nitems` elements, then randomly samples elements from this buffer, replacing the selected elements with new elements. Be sure to specify a large enough buffer size. If one calls `repeat()` on a shuffled dataset, a new order will be generated every iteration. To preserve the same order (e.g. for testing), set `reshuffle_each_iteration=False`.

Pipelining Transformations

The `tf.data.Dataset` API supports all sorts of transformations, enabling the construction of efficient and descriptive pipelines. Each transformation returns a new dataset. The usage follows this common pattern:

- 1 Create a source dataset from data (stored in memory or in disk).
- 2 Create new datasets by applying data processing transformations.
- 3 Use the last created dataset to iterate and process its elements

Some Functions for Pipeline Execution

```
DT=tf.data.Dataset.range(10).repeat(2).batch(6)
for i in DT:
    print(i)
```

output

```
tf.Tensor([0 1 2 3 4 5], shape=(6,), dtype=int64)
tf.Tensor([6 7 8 9 0 1], shape=(6,), dtype=int64)
tf.Tensor([2 3 4 5 6 7], shape=(6,), dtype=int64)
tf.Tensor([8 9], shape=(2,), dtype=int64)
```

To drop the last batch use `batch(6, drop_remainder=True)`.
This way all the batches have the same size.

Pipelining Transformations

The `tf.data.Dataset` API supports all sorts of transformations, enabling the construction of efficient and descriptive pipelines. Each transformation returns a new dataset. The usage follows this common pattern:

- 1 Create a source dataset from data (stored in memory or in disk).
- 2 Create new datasets by applying data processing transformations.
- 3 Use the last created dataset to iterate and process its elements

Some Functions for Pipeline Execution

```
DT=tf.data.Dataset.range(10).batch(6).repeat(2)
for i in DT:
    print(i)
```

output

```
tf.Tensor([0 1 2 3 4 5], shape=(6,), dtype=int64)
tf.Tensor([6 7 8 9], shape=(4,), dtype=int64)
tf.Tensor([0 1 2 3 4 5], shape=(6,), dtype=int64)
tf.Tensor([6 7 8 9], shape=(4,), dtype=int64)
```

Be aware that calling the functions in a different order yields different results!

Pipelining Transformations

The `tf.data.Dataset` API supports all sorts of transformations, enabling the construction of efficient and descriptive pipelines. Each transformation returns a new dataset. The usage follows this common pattern:

- 1 Create a source dataset from data (stored in memory or in disk).
- 2 Create new datasets by applying data processing transformations.
- 3 Use the last created dataset to iterate and process its elements

Some Functions for Pipeline Execution

```
DT=tf.data.Dataset.range(5).shuffle(buffer_size=5).repeat(3).batch(6)
for i in DT:
    print(i)
```

output

```
tf.Tensor([0 3 1 2 4 4], shape=(6,), dtype=int64)
tf.Tensor([1 2 0 3 4 1], shape=(6,), dtype=int64)
tf.Tensor([0 3 2], shape=(3,), dtype=int64)
```

In the `shuffle` method the buffer size must be specified. Ideally a buffer equal or greater than the total number of elements in the dataset is needed for a perfect shuffle... but be careful not to exceed the available amount of RAM!

Pipelining Transformations

The `tf.data.Dataset` API supports all sorts of transformations, enabling the construction of efficient and descriptive pipelines. Each transformation returns a new dataset. The usage follows this common pattern:

- 1 Create a source dataset from data (stored in memory or in disk).
- 2 Create new datasets by applying data processing transformations.
- 3 Use the last created dataset to iterate and process its elements

Some Functions for Pipeline Execution

```
DT=tf.data.Dataset.range(5).repeat(3).batch(6).shuffle(buffer_size=5)
for i in DT:
    print(i)
```

output

```
tf.Tensor([0 1 2 3 4 0], shape=(6,), dtype=int64)
tf.Tensor([2 3 4], shape=(3,), dtype=int64)
tf.Tensor([1 2 3 4 0 1], shape=(6,), dtype=int64)
```

Be sure to use `shuffle` before calling the `batch` method!!!

Pipelining Transformations

The `tf.data.Dataset` API supports all sorts of transformations, enabling the construction of efficient and descriptive pipelines. Each transformation returns a new dataset. The usage follows this common pattern:

- 1 Create a source dataset from data (stored in memory or in disk).
- 2 Create new datasets by applying data processing transformations.
- 3 Use the last created dataset to iterate and process its elements

Some Functions for Data Transformation

- `map` method:

The method `map(fun)` produces a new dataset by applying a given function `fun()` to each element of the original dataset. This method should only use TensorFlow objects and operations. This has to do with the way TensorFlow works under the hood. First it analyses the computations performed by `fun()` and generates an equivalent computational graph. To convert Python functions to TensorFlow functions, one can use `tf.function()` - this converts generic functions into TensorFlow graphs. Alternatively, one can use `tf.py_function()` - more on this later.

- `apply` method:

While the `map()` methods applies a transformation to each item, the `apply()` method applies the transformation to the whole dataset.

Pipelining Transformations

The `tf.data.Dataset` API supports all sorts of transformations, enabling the construction of efficient and descriptive pipelines. Each transformation returns a new dataset. The usage follows this common pattern:

- 1 Create a source dataset from data (stored in memory or in disk).
- 2 Create new datasets by applying data processing transformations.
- 3 Use the last created dataset to iterate and process its elements

Some Functions for Data Transformation

```
DT=tf.data.Dataset.range(10).repeat(3).batch(5)
DT=DT.map(lambda x :x*(x%2)) # only odd numbers
for i in DT:
    print(i.numpy())
```

output

```
[0 1 0 3 0]
[5 0 7 0 9]
[0 1 0 3 0]
[5 0 7 0 9]
[0 1 0 3 0]
[5 0 7 0 9]
```

Pipelining Transformations

The `tf.data.Dataset` API supports all sorts of transformations, enabling the construction of efficient and descriptive pipelines. Each transformation returns a new dataset. The usage follows this common pattern:

- 1 Create a source dataset from data (stored in memory or in disk).
- 2 Create new datasets by applying data processing transformations.
- 3 Use the last created dataset to iterate and process its elements

Some Functions for Data Transformation

or...

```
def odd(x):  
    return x*(x%2)  
DT=tf.data.Dataset.range(10).repeat(3).batch(5)  
DT=DT.map(odd) # only odd numbers  
for i in DT:  
    print(i.numpy())
```

output

```
[0 1 0 3 0]  
[5 0 7 0 9]  
[0 1 0 3 0]  
[5 0 7 0 9]  
[0 1 0 3 0]  
[5 0 7 0 9]
```

Pipelining Transformations

The `tf.data.Dataset` API supports all sorts of transformations, enabling the construction of efficient and descriptive pipelines. Each transformation returns a new dataset. The usage follows this common pattern:

- 1 Create a source dataset from data (stored in memory or in disk).
- 2 Create new datasets by applying data processing transformations.
- 3 Use the last created dataset to iterate and process its elements

Some Functions for Data Transformation

or...

```
def odd(x):  
    return tf.math.multiply(x,tf.math.floormod(x,2))  
DT=tf.data.Dataset.range(10).repeat(3).batch(5)  
DT=DT.map(odd) # only odd numbers  
for i in DT:  
    print(i.numpy())
```

output

```
[0 1 0 3 0]  
[5 0 7 0 9]  
[0 1 0 3 0]  
[5 0 7 0 9]  
[0 1 0 3 0]  
[5 0 7 0 9]
```


Pipelining Transformations

The `tf.data.Dataset` API supports all sorts of transformations, enabling the construction of efficient and descriptive pipelines. Each transformation returns a new dataset. The usage follows this common pattern:

- 1 Create a source dataset from data (stored in memory or in disk).
- 2 Create new datasets by applying data processing transformations.
- 3 Use the last created dataset to iterate and process its elements

Some Functions for Data Transformation

BUT NOT:

```
def odd(x):  
    return x*(np remainder(x,2)) # Numpy functions are not allowed!!!  
DT=tf.data.Dataset.range(10).repeat(3).batch(5)  
DT=DT.map(odd)
```

output

`NotImplementedError: Cannot convert a symbolic Tensor (args_0:0) to a numpy array. This error may indicate that you're trying to pass a Tensor to a NumPy call, which is not supported`

Pipelining Transformations

The `tf.data.Dataset` API supports all sorts of transformations, enabling the construction of efficient and descriptive pipelines. Each transformation returns a new dataset. The usage follows this common pattern:

- 1 Create a source dataset from data (stored in memory or in disk).
- 2 Create new datasets by applying data processing transformations.
- 3 Use the last created dataset to iterate and process its elements

Performance Enhancing Functions and Parameters

Achieving an enhanced performance requires an efficient input pipeline so that data is delivered for the next step (to CPUs, GPUs, or TPUs) before the current step has finished. Here are a few methods designed for performance optimization.

(Also check out [Better Performance with the `tf.data` API](#)).

- `prefetch` method:
By calling `prefetch(n)` at the end, the resulting dataset will try to be `n` batches ahead. While the algorithms are working on a given batch, the other batches are being worked in parallel (e.g. reading data from disk and preprocessing it).
- `cache` method:
If the dataset is small enough to fit in memory, one can use the `cache` method to speed up training by caching its contents in RAM. One should do this after loading and preprocessing the data and before `shuffle`, `repeat`, `batch` and `prefetch`.

Pipelining Transformations

The `tf.data.Dataset` API supports all sorts of transformations, enabling the construction of efficient and descriptive pipelines. Each transformation returns a new dataset. The usage follows this common pattern:

- 1 Create a source dataset from data (stored in memory or in disk).
- 2 Create new datasets by applying data processing transformations.
- 3 Use the last created dataset to iterate and process its elements

Performance Enhancing Functions and Parameters

Achieving an enhanced performance requires an efficient input pipeline so that data is delivered for the next step (to CPUs, GPUs, or TPUs) before the current step has finished. Here are a few methods designed for performance optimization.

(Also check out [Better Performance with the `tf.data` API](#)).

- `num_parallel_calls` parameter:
This parameter used in `map` and `interleave` methods, ensures that the operations are multi-threaded.

TensorFlow Functions and Graphs

There are two options for running code in TensorFlow:

- **Eager Execution Mode:**

In eager execution mode, instructions are executed immediately. This is what is expected with standard Python code. Instructions are executed line by line and computational results are returned immediately. This makes coding and debugging easier.

- **Graph Mode:**

In graph mode, TensorFlow analyses Python functions and converts them into computational graphs, optimizing them through pruning unused nodes, simplifying expressions, and more. Once it has done that, operations can be executed efficiently (e.g. via parallel computations on GPU, CPU or TPU cores), and as a result, TF functions will generally run much faster than the standard Python functions, especially when it performs complex computations.

In the context of data generators, particularly when implementing data transformation functions, it is important to know the basic rules for running TF in **graph mode**. In this section we will cover the changes that code needs so that data generators can run in graph mode.

TensorFlow Functions and Graphs

To convert a regular function into a TensorFlow function so it can run in graph mode, one can use the command `tf.function()` or decorate the function with `@tf.function`.

Example:

```
@tf.function
def odd(x):
    print(x)
    return x*(x%2)

DT=tf.data.Dataset.range(10).shuffle(10).batch(5).repeat(1)
DT=DT.map(odd)
[print(i) for i in DT]
print("2nd run:")
[print(i) for i in DT]
```

output

```
Tensor("x:0", shape=(None,), dtype=int64)
tf.Tensor([5 7 0 9 1], shape=(5,), dtype=int64)
tf.Tensor([0 0 0 0 3], shape=(5,), dtype=int64)
2nd run:
tf.Tensor([0 9 0 0 0], shape=(5,), dtype=int64)
tf.Tensor([7 1 3 0 5], shape=(5,), dtype=int64)
```

TensorFlow Functions and Graphs

To convert a regular function into a TensorFlow function so it can run in graph mode, one can use the command `tf.function()` or decorate the function with `@tf.function`.

Example:

```
@tf.function
def odd(x):
    print(x)
    return x*(x%2)

DT=tf.data.Dataset.range(10).shuffle(10).batch(5).repeat(1)
DT=DT.map(odd)
[print(i) for i in DT]
print("2nd run:")
[print(i) for i in DT]
```

Note that the `print(x)` command inside the the function `odd(x)` is only executed once, and the variable `x` that is printed has no specific value:

```
Tensor("x:0", shape=(None,), dtype=int64)
```

This has to do with the way TensorFlow converts code into graph mode. When the function is first invoked, the function is **traced**: a computational graph is created, and operations are deferred. The operations only happen after the graph is created. In this particular case, the `print(x)` statement is executed during tracing only, but it does not add any nodes to the graph. This operation is phrased as a **Python side effect** as it has impacts during tracing but not during the graph execution.

TensorFlow Functions and Graphs

To convert a regular function into a TensorFlow function so it can run in graph mode, one can use the command `tf.function()` or decorate the function with `@tf.function`.

Example:

```
@tf.function
def odd(x):
    tf.print(x)
    return x*(x%2)
DT=tf.data.Dataset.range(10).shuffle(10).batch(5).repeat(1)
DT=DT.map(odd)
[print(i) for i in DT]
```

output

```
[4 9 1 0 5]
tf.Tensor([0 9 1 0 5], shape=(5,), dtype=int64)
[2 8 6 7 3]
tf.Tensor([0 0 0 7 3], shape=(5,), dtype=int64)
```

On the contrary to `print(x)`, `tf.print(x)` is a TF operation. It adds a node to the graph in the tracing without any output to the console. It prints out to the console when the graph is executed.

For a better understanding of how tracing and graph execution work, check out the TensorFlow manual page: [Better performance with `tf.function`](#)

TensorFlow Functions and Graphs

Executing Python Code in Graphs

In the case we need to run Python code in a graph, TensorFlow provides the command `tf.py_function` as a workaround. Keep in mind that the benefits of the graph mode will be lost. Therefore one should avoid using this workaround whenever possible. Nevertheless, there are cases that one might need to take advantage of specific Python libraries functionalities that are not yet available in TensorFlow (e.g., image data augmentation, audio data augmentation, database querying , etc).

Example:

```
def odd_numpy(x):  
    print(x)  
    return x*np remainder(x,2)  
def wrapper(x):  
    return tf.py_function(func=odd_numpy,inp=[x],Tout=[tf.int64])  
DT=tf.data.Dataset.range(10).shuffle(10).batch(5).repeat(1)  
DT=DT.map(wrapper)  
[print(i) for i in DT]
```

output

```
tf.Tensor([4 1 0 6 5], shape=(5,), dtype=int64)  
(<tf.Tensor:  shape=(5,), dtype=int64, numpy=array([0, 1, 0, 0, 5])>,)  
tf.Tensor([3 2 8 9 7], shape=(5,), dtype=int64)  
(<tf.Tensor:  shape=(5,), dtype=int64, numpy=array([3, 0, 0, 9, 7])>,) 
```


TensorFlow Functions and Graphs

Executing Python Code in Graphs

In the case we need to run Python code in a graph, TensorFlow provides the command `tf.py_function` as a workaround. Keep in mind that the benefits of the graph mode will be lost. Therefore one should avoid using this workaround whenever possible. Nevertheless, there are cases that one might need to take advantage of specific Python libraries functionalities that are not yet available in TensorFlow (e.g., image data augmentation, audio data augmentation, database querying , etc).

Example:

```
def odd_numpy(x):  
    print(x)  
    return x*np remainder(x,2)  
def wrapper(x):  
    return tf.py_function(func=odd_numpy,inp=[x],Tout=[tf.int64])  
DT=tf.data.Dataset.range(10).shuffle(10).batch(5).repeat(1)  
DT=DT.map(wrapper)  
[print(i) for i in DT]
```

Note that `tf.py_function` takes three input arguments: 1) the name of the Python function, 2) the input arguments to the Python function, 3) the type of the value(s) returned by the Python function. In this particular case, since the inputs are produced by a generator (and not yet instantiated), one needs a wrapper function to circumvent this problem. Also note that the code is run in eager mode: the `print()` is executed in every call.

TensorFlow Functions and Graphs

TensorFlow Function Rules

- Use TensorFlow operations as a rule. If one calls standard or external Python libraries commands, these will not be part of the graph and are only executed during tracing. For example, use `tf.print()` instead of `print()`, use `for i in tf.range(n)` instead of `for i in range(n)`, use `tf.sort()` instead of `np.sort()`, and so on and so forth.
- If a function creates a TensorFlow variable, it must do so upon the very first call or an exception will be raised. It is generally preferable to create variables outside the TensorFlow function.
- If a TensorFlow function is called with different values (e.g. different shapes or types) it is retraced adding overhead. In particular, if scalars are used, the retrace process is triggered whenever the values are different.
- The source code of the function must be available to TensorFlow. If that is not the case (e.g. if the function is defined in the Python shell), the graph generation process will fail or have limited functionality.
- Wrapping arbitrary Python code with a `tf.py_function` operation will hinder performance since TensorFlow will not be able to do any optimization on this code.
- As a general rule to boost performance, one should prefer vectorized implementations rather than using loops.

Class Sub-Folder Structure

When images are pre-split into class sub-folders,

```
keras.utils.image_dataset_from_directory
```

builds a batched `tf.data` pipeline in one call.

Example: `tf_flowers`

5 classes, pre-split into `train/`, `validation/` and `test/` directories

```
tf_flowers/  
  train/  
    daisy/  dandelion/  roses/  sunflowers/  tulips/  
  validation/  daisy/  ...  
  test/        daisy/  ...
```

- Sub-folder names *are* the class names: no annotation file is needed.
- Labels are assigned in alphabetical order of folder names:
daisy → 0, dandelion → 1, roses → 2, sunflowers → 3, tulips → 4.
- Use `dataset.class_names` to retrieve the ordered name list at any time.

Class Sub-Folder Structure

When images are pre-split into class sub-folders,

```
keras.utils.image_dataset_from_directory
```

builds a batched `tf.data` pipeline in one call.

Example: `tf_flowers`

5 classes, pre-split into `train/`, `validation/` and `test/` directories

```
IMG_SIZE = (224, 224)
```

```
BATCH = 32
```

```
trainGen = keras.utils.image_dataset_from_directory(  
    'tf_flowers/train',  
    image_size = IMG_SIZE,  
    batch_size = BATCH,  
    label_mode = 'int',  
    shuffle = True,  
    seed = 42)
```

```
print(trainGen.class_names)
```

output

Found 2569 files belonging to 5 classes.

```
['daisy', 'dandelion', 'roses', 'sunflowers', 'tulips']
```

The returned dataset is already **batched**.

Each element is a tuple (`image_batch`, `label_batch`).

Class Sub-Folder Structure: Key Parameters

- `image_size=(H,W)`: all images are resized to this shape on load. Must match the input shape expected by the model.
- `batch_size`: number of images per returned batch.
- `shuffle=True` (training only): file list is reshuffled before each epoch so batches change every epoch. Use `shuffle=False` for validation and test to keep evaluation order fixed. Always set `seed` for reproducibility.
- `label_mode`:
 - `'int'`: integer labels;
pair with `sparse_categorical_crossentropy`.
 - `'categorical'`: one-hot vectors;
pair with `categorical_crossentropy`.
 - `'binary'`: single float (0 or 1);
pair with `binary_crossentropy`.

Class Sub-Folder Structure: Key Parameters

Recommended order of pipeline calls:

The function `image_dataset_from_directory` returns a **batched** dataset.

No further `.batch()` or `.shuffle()` calls are needed.

Chain the remaining steps in this order:

- 1 `.cache()`: after the first epoch, loaded image batches are kept in RAM so disk reads are not repeated.
Place **before** `.map()` when data augmentation is used, so that fresh random transforms are applied every epoch from the cached raw images.
- 2 `.map(fn, num_parallel_calls=AUTOTUNE)`: apply preprocessing and/or augmentation to each batch.
- 3 `.prefetch(AUTOTUNE)`: overlap preparation of batch $n + 1$ with model execution on batch n , hiding I/O latency.

Class Sub-Folder Structure: The `map()` Function

Each element of the dataset is a **tuple** (`image_batch`, `label_batch`). The `map()` function is applied element-wise and receives both tuple components.

One example is chaining the image pre-processing function of a given pre-trained CNN to the generator. Note that in this case, only the images are modified, and the label batch must be returned unchanged: `model.fit()` expects (X, y) tuples throughout the pipeline.

```
from keras.applications.mobilenet_v2 import preprocess_input  
def prep(imgs, labels): # labels passed through, images normalised  
    return preprocess_input(imgs), labels
```

`preprocess_input` for `MobileNetV2` maps pixel values from $[0, 255]$ to $[-1, 1]$.
`image_dataset_from_directory` already returns `float32` images, so no explicit cast is needed here.

Class Sub-Folder Structure: The `map()` Function

Data Augmentation:

Define a `Sequential` augmentation model using Keras preprocessing layers and apply it inside the `map()` function for the training pipeline only.

```
data_aug = tf.keras.Sequential([
    tf.keras.layers.RandomFlip('horizontal'),
    tf.keras.layers.RandomRotation(30/360),
    tf.keras.layers.RandomTranslation(0.2, 0.2),
    tf.keras.layers.RandomZoom(0.2),
])

def prep_aug(imgs, labels):
    imgs = data_aug(imgs, training=True)
    return preprocess_input(imgs), labels
```

- `training=True` activates the random transforms on each call.
- `prep_aug`: augments first on raw `[0, 255]` pixel values, then normalises. Used for **training only**.
- `prep`: normalises only, no augmentation. Used for **validation and test**: augmenting evaluation data would give inconsistent metrics.

Class Sub-Folder Structure

`cache()`, `AUTOTUNE`, and `prefetch()`

- `tf.data.AUTOTUNE` (value `-1`) tells TensorFlow to tune the number of parallel worker threads automatically at runtime. Pass it to `num_parallel_calls` in `.map()` and to `.prefetch()`.
- `.cache()` placed **before** `.map(prepare_aug)`: raw image batches are cached; augmentation runs fresh every epoch with new random transforms.
`.cache()` placed **after** `.map(prepare_aug)`: augmented results are stored; the same transforms repeat every epoch. **Not recommended for training.**
- `.prefetch(AUTOTUNE)` should always be the **last** step in the pipeline: it prepares the next batch in the background while the model processes the current one.

Class Sub-Folder Structure: Full Pipeline

Training generator

```
AUTOTUNE = tf.data.AUTOTUNE

trainGen = keras.utils.image_dataset_from_directory(
    'tf_flowers/train', image_size=IMG_SIZE,
    batch_size=BATCH, label_mode='int',
    shuffle=True, seed=42)
trainGen = (trainGen
    .cache()
    .map(prepare_aug, num_parallel_calls=AUTOTUNE)
    .prefetch(tf.keras.Sequential([
        tf.keras.layers.RandomFlip('horizontal'),
        tf.keras.layers.RandomRotation(30/360),
        tf.keras.layers.RandomTranslation(0.2, 0.2),
        tf.keras.layers.RandomZoom(0.2),
    ]))

def prepare_aug(imgs, labels):
    imgs = data_aug(imgs, training=True)
    return preprocess_input(imgs), labels

ch(AUTOTUNE))
```

`cache()` stores raw image batches after the first epoch; `prepare_aug` then runs fresh augmentation every epoch.

Note: caching 2569 images at $224 \times 224 \times 3$ float32 needs ≈ 1.2 GB of RAM. If RAM is limited,

emit `tf.data.Options.experimental_preload_images = False` and images are reloaded from disk each epoch.

Class Sub-Folder Structure: Full Pipeline

Validation generator (test follows the same pattern)

```
validGen = keras.utils.image_dataset_from_directory(  
    'tf-flowers/validation', image_size=IMG_SIZE,  
    batch_size=BATCH, label_mode='int',  
    shuffle=False)  
validGen = (validGen  
    .map(prepare, num_parallel_calls=AUTOTUNE)  
    .cache()  
    .prefetch(AUTOTUNE))
```

For validation/test: `shuffle=False` keeps evaluation order fixed. `.map(prepare)` runs before `.cache()` since preprocessing is deterministic. Preprocessed batches are stored and reused every epoch at no extra cost.

Pass to the model: `cnn.fit(trainGen, validation_data=validGen, epochs=25)`

Generators from In-Memory Arrays

When data is already loaded into NumPy arrays (e.g. CIFAR-10),

```
tf.data.Dataset.from_tensor_slices()
```

wraps them in a `tf.data` pipeline, adding shuffle, batching, and preprocessing on top.

Loading CIFAR-10 and defining the preprocessing function

```
(X_train, y_train), (X_test, y_test) =  
    keras.datasets.cifar10.load_data()  
y_train = y_train.squeeze() # (50000,1) → (50000,)  
y_test = y_test.squeeze()
```

```
def prep(img, label):  
    return tf.cast(img, tf.float32) / 255.0, label
```

`tf.cast` converts `uint8` values (0–255) to `float32` before dividing. The label is returned unchanged. The preprocessing function must always return the full `(X, y)` tuple.

Generators from In-Memory Arrays

When data is already loaded into NumPy arrays (e.g. CIFAR-10),

```
tf.data.Dataset.from_tensor_slices()
```

wraps them in a `tf.data` pipeline, adding shuffle, batching, and preprocessing on top.

Pipeline and recommended ordering

```
AUTOTUNE = tf.data.AUTOTUNE
```

```
trainGen = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
            .shuffle(X_train.shape[0], seed=42)
            .batch(32)
            .map(prepare, num_parallel_calls=AUTOTUNE)
            .prefetch(AUTOTUNE))
```

```
testGen = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
           .batch(32)
           .map(prepare, num_parallel_calls=AUTOTUNE)
           .prefetch(AUTOTUNE))
```

- **No** `cache()`: arrays already live in RAM; caching would duplicate memory.
- **shuffle before batch**: shuffle individual samples for unbiased batches.
- **map after batch**: preprocessing runs on whole batches (vectorised, faster).
- Buffer equal to dataset size gives a perfect shuffle; reduce if RAM is limited.

Generators from In-Memory Arrays

Data Preparation:

When fine-tuning a pre-trained backbone (e.g. MobileNetV2), replace the generic `/255` with the backbone-specific `preprocess_input`.

```
from keras.applications.mobilenet_v2 import preprocess_input

def prep(imgs, labels):
    imgs = tf.cast(imgs, tf.float32)
    return preprocess_input(imgs), labels
```

- `preprocess_input` for MobileNetV2 expects `float32` in `[0, 255]` and maps to `[-1, 1]`. Do **not** divide by 255 first. The function does its own normalisation.
- The `labels` argument is returned unchanged. Every preprocessing function in a `(X, y)` pipeline must return both components. `model.fit()` requires the full tuple at every step.
- Each backbone has its own `preprocess_input`: always import it from the matching module (e.g. `keras.applications.vgg16.preprocess_input` for VGG16).

Generators from In-Memory Arrays

Data Preparation:

One can also add data augmentation to the training phase.

```
data_aug = tf.keras.Sequential([
    tf.keras.layers.RandomFlip('horizontal'),
    tf.keras.layers.RandomRotation(30/360),
    tf.keras.layers.RandomTranslation(0.2, 0.2),
    tf.keras.layers.RandomZoom(0.2),
])
```

```
def prep_aug(imgs, labels):
    imgs = data_aug(imgs, training=True)
    return preprocess_input(imgs), labels
```

- `training=True` activates the random transforms on each call.
- `prep_aug`: augments first on raw `[0,255]` pixel values, then normalizes. Used for **training only**.
- `prep`: normalizes only, no augmentation. Used for **validation and test**: augmenting evaluation data would give inconsistent metrics.

Generators from In-Memory Arrays

Pipelines for training, validation and test

```
# split training data to obtain a validation set
X_tr, X_val, y_tr, y_val = train_test_split(
    X_train, y_train, test_size=0.15, stratify=y_train)

trainGen = (tf.data.Dataset.from_tensor_slices((X_tr, y_tr))
            .shuffle(X_tr.shape[0], seed=42)
            .batch(32)
            .map(prepare, num_parallel_calls=AUTOTUNE)
            .prefetch(AUTOTUNE))

validGen = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
            .batch(32)
            .map(prepare, num_parallel_calls=AUTOTUNE)
            .prefetch(AUTOTUNE))

testGen follows the same pattern as validGen using X_test, y_test.
validGen and testGen have no .shuffle(): evaluation order must be fixed.
```


Image Data Pipelines: single directory

`image_dataset_from_directory` requires images organised in class sub-folders:

```
train/class_a/img1.jpg  train/class_b/img2.jpg  ...
```

Running Example: Oxford-IIIT-Pet Dataset

- 37 breeds (25 dogs, 12 cats), 7349 images total.
- All images in a single flat directory `images/`.
- Labels in `annotations/list.txt`, one row per image:

```
Abyssinian_1    1    1    1
```

```
fields: image_name  class_id  species_id  breed_id
```

- The class name is also encoded in the filename prefix (before the last `_N`).

Image Data Pipelines: single directory

`image_dataset_from_directory` requires images organised in class sub-folders:

```
train/class_a/img1.jpg  train/class_b/img2.jpg  ...
```

Approach

- 1 Parse `annotations/list.txt` into a Pandas DataFrame.
- 2 Use `train_test_split` (sklearn) to divide file paths and labels into train / validation / test lists, stratified by class.
- 3 Build one `tf.data.Dataset` per subset with `from_tensor_slices((paths, labels))`.
- 4 Apply `map()` to load images on demand from disk.
- 5 Apply `map()` again for preprocessing and augmentation.
- 6 Apply `batch()` and `prefetch()` for training efficiency.

The `map()` functions that load and preprocess images are **identical** regardless of how the labels were obtained.

Reading the Annotation File

annotations/list.txt **begins with comment lines (#) followed by one row per image.**

```
import pandas as pd
df = pd.read_csv('annotations/list.txt',
                 sep=' ', comment='#', header=None,
                 names=['name', 'class_id', 'species', 'breed_id'])
df['filepath'] = 'images/' + df['name'] + '.jpg'
df['label'] = df['class_id'] - 1
df['class_name'] = df['name'].str.rsplit('_', n=1).str[0]
print(df[['filepath', 'class_name', 'label']].head())
```

NOTE: `rsplit('_', n=1)` splits on the **last** underscore only, so multi-word breeds (e.g. `British_Shorthair`) are handled correctly.

Reading the Annotation File

`annotations/list.txt` begins with comment lines (`#`) followed by one row per image.

Output of `df.head()`:

filepath	class_name	label
images/Abyssinian_1.jpg	Abyssinian	0
images/Bengal_1.jpg	Bengal	1
images/Birman_1.jpg	Birman	2
images/Bombay_1.jpg	Bombay	3
images/British_Shorthair_1.jpg	British_Shorthair	4

37 unique class names; integer labels 0–36; 7349 rows total.

Splitting with `train_test_split`

Using `sklearn.model_selection.train_test_split` on the file-path arrays gives a stratified, reproducible split and produces one `tf.data.Dataset` per subset directly.

```
from sklearn.model_selection import train_test_split

paths = df['filepath'].values
labels = df['label'].values.astype('int32')

# step 1: 70% train, 30% temp
paths_tr, paths_tmp, labels_tr, labels_tmp = train_test_split(
    paths, labels, test_size=0.30,
    stratify=labels, random_state=42)

# step 2: split the 30% equally → val and test (15% each)
paths_val, paths_te, labels_val, labels_te = train_test_split(
    paths_tmp, labels_tmp, test_size=0.50,
    stratify=labels_tmp, random_state=42)

train_ds = tf.data.Dataset.from_tensor_slices((paths_tr, labels_tr))
val_ds = tf.data.Dataset.from_tensor_slices((paths_val, labels_val))
test_ds = tf.data.Dataset.from_tensor_slices((paths_te, labels_te))

stratify=labels ensures each split preserves the class proportions across all 37 breeds. At
this point each dataset yields (path-string, integer-label) pairs; images are not yet
loaded.
```

Loading and Preprocessing Images

A `map()` function converts each `(path, label)` pair into an `(image, label)` pair.

```
from keras.applications.vgg16 import preprocess_input
IMG_SIZE = (224, 224)

@tf.function
def load_img(path, label):
    raw = tf.io.read_file(path)
    img = tf.image.decode_jpeg(raw, channels=3)
    img = tf.image.resize(img, IMG_SIZE)
    return img, label

def preproc(img, label):
    return preprocess_input(img), label
```

Loading and Preprocessing Images

A `map()` function converts each `(path, label)` pair into an `(image, label)` pair.

- `tf.io.read_file` reads raw bytes from disk.
- `tf.image.decode_jpeg` decodes to a `uint8` tensor.
- `tf.image.resize` resizes and promotes to `float32` in `[0,255]`.
- `preprocess_input` (VGG16 module) subtracts the ImageNet per-channel means and converts `RGB` $\rightarrow$ `BGR`, matching what the pre-trained weights expect.
- The `@tf.function` decorator compiles `loading` into a TF graph, enabling multi-threaded loading via `num_parallel_calls`.

Loading and Preprocessing Images

A `map()` function converts each `(path, label)` pair into an `(image, label)` pair.

- **Why decorate `load_img` with `@tf.function`?**

- ▶ The decorator compiles the function into a static TF graph: Python tracing runs once; afterwards TF dispatches native ops with no per-element Python overhead.
- ▶ `load_img` is the I/O bottleneck (disk reads + JPEG decode). With `num_parallel_calls=tf.data.AUTOTUNE`, multiple compiled graph instances run concurrently in separate threads.

- **Why is `@tf.function` *not* needed on `preproc` / `preproc_aug`?**

- ▶ Both operate on tensors already in memory: no I/O, no decode step.
- ▶ `preprocess_input` is pure tensor arithmetic (mean subtraction + channel swap): fast enough in eager mode; graph compilation adds no benefit here.
- ▶ `data_aug` is a Keras `Sequential` model whose `call()` is already compiled to a graph internally by Keras. An outer `@tf.function` would be redundant and can cause re-tracing as the `training` argument changes.

Data Augmentation

Define a `Sequential` augmentation model; apply it only to the training pipeline.

```
data_aug = tf.keras.Sequential([
    tf.keras.layers.RandomFlip('horizontal'),
    tf.keras.layers.RandomRotation(30/360),
    tf.keras.layers.RandomTranslation(0.2, 0.2),
    tf.keras.layers.RandomZoom(0.2),
])

def preproc_aug(img, label):
    img = data_aug(img, training=True)
    return preprocess_input(img), label
```

- `preproc_aug`: augments on raw `[0,255]` pixel values, then normalizes. Used for training only.
- `preproc`: normalizes only (defined on the previous slide). Used for validation and test.
- **Do not augment validation or test data**: this distorts evaluation results.

The Full Pipeline

Assemble the training pipeline. The position of `cache()` and `shuffle()` matters:

```
BATCH = 32
AUTOTUNE = tf.data.AUTOTUNE

train_ds = (train_ds
            .map(load_img, num_parallel_calls=AUTOTUNE)
            .cache()
            .shuffle(1000)
            .map(preproc_aug, num_parallel_calls=AUTOTUNE)
            .batch(BATCH)
            .prefetch(AUTOTUNE))
```

`cache()` is placed **after** `load_img` but **before** `preproc_aug`:

- loaded images are stored in RAM and reused across epochs (fast);
- augmentation runs **fresh every epoch** (new random transforms each time).

Warning: caching 7349 images at $224 \times 224 \times 3$ float32 needs ≈ 3 GB of RAM.

The Full Pipeline

Validation and test pipelines (no augmentation):

```
val_ds = (val_ds
          .map(load_img, num_parallel_calls=AUTOTUNE)
          .map(preproc, num_parallel_calls=AUTOTUNE)
          .cache().batch(BATCH).prefetch(AUTOTUNE))

test_ds = (test_ds
           .map(load_img, num_parallel_calls=AUTOTUNE)
           .map(preproc, num_parallel_calls=AUTOTUNE)
           .cache().batch(BATCH).prefetch(AUTOTUNE))
```

Pass the datasets directly to `model.fit()`: no `steps_per_epoch` needed:

```
cnn.fit(train_ds, validation_data=val_ds, epochs=50)
```

One epoch = one full pass through all 5144 training images = $\lceil 5144/32 \rceil = 161$ batches.